



CHAPTER 12

Advanced Python 1



Following are some of the newly added features in Python programming language.

1. Walrus Operator

The walrus operator (`:=`), introduced in Python 3.8, allows you to assign values to variables as part of an expression.



Using Walrus operator

```
if (n := len([1, 2, 3, 4, 5])) > 3:  
    print(f"List is too long ({n} elements, expected <= 3)")
```



In this example, `n` is assigned the value of `len([1, 2, 3, 4, 5])` and then used in the comparison within the `if` statement.

2. Types Definitions In Python

Type hints are added using the colon (`:`) syntax for variables and the `->` syntax for function return types.

Variable type hint

```
age: int = 25
```

Function type hints

```
def greeting(name: str) -> str:  
    return f"Hello, {name}!"
```

Usage

```
print(greeting("Alice"))
```

Type Hints

- Make code more readable
- Help in catching type related errors
- Useful for IDEs and documentation



3. Advanced Type Hints

Python's typing module provides more advanced type hints, such as `List`, `Tuple`, `Dict` and `Union`.

```
from typing import List, Tuple, Dict, Union
```

List of integers

```
numbers: List[int] = [1, 2, 3, 4, 5]
```

Tuple of a string and an integer

```
person: Tuple[str, int] = ("Alice", 30)
```

Dictionary with string keys and integer values

```
scores: Dict[str, int] = {"Alice": 90, "Bob": 85}
```

Union type

```
identifier: Union[int, str] = "ID123"
```

Common Types

- `List[T]` → list of type `T`
- `Tuple[T1, T2]` → tuple
- `Dict[K, V]` → dictionary
- `Union[A, B]` → `A` or `B`



4. Match Case

Python 3.10 introduced the match statement, which is similar to the switch statement found in other programming languages.

```
def http_status(status):  
    match status:  
        case 200:  
            return "OK"  
        case 404:  
            return "Not Found"  
        case 500:  
            return "Internal Server Error"  
        case _:  
            return "Unknown status"
```

```
print(http_status(200))  
print(http_status(404))
```

match is a more readable way to handle multiple conditions.

How it works?

- match evaluates the given value.
- case checks for a pattern match.
- case _ (underscore) acts as the default case.

It improves code readability when dealing with many possible values.

5. Dictionary Merge & Update Operators

New operators | and |= allow for merging and updating dictionaries.

```
dict1 = {'a': 1, 'b': 2}  
dict2 = {'b': 3, 'c': 4}  
  
merged = dict1 | dict2  
print(merged)  
# Output: {'a': 1, 'b': 3, 'c': 4}
```

dict1

```
{'a': 1,  
'b': 2}
```

dict2

```
{'b': 3,  
'c': 4}
```

merged

```
{'a': 1,  
'b': 3,  
'c': 4}
```

If keys are same, the right dictionary value wins!

Notes:

- `d1 | d2` → merges two dictionaries (new dict).
- `d1 |= d2` → updates d1 with d2 (in-place).

6. with Statement With Multiple Context Managers

You can handle multiple context managers in a single with statement using parentheses.

```
with (  
    open('file1.txt') as f1,  
    open('file2.txt') as f2  
):  
    # Process files  
    pass
```

Benefits:

- Cleaner and more readable.
- Both files are properly closed even if an error occurs.
- Equivalent to nesting with statements.



file1.txt



file2.txt

Great for working with multiple files, connections, etc.



Exception Handling In Python

There are many built-in exceptions which are raised in python when something goes wrong.



1. Basic Exception Handling

```
try:
    # Code which might throw exception
    print(10 / 0)
except Exception as e:
    print(e)
```

What happens?

Dividing by zero raises an exception. The except block catches it and prints the error message.



2. Multiple Except Blocks

```
try:
    # Code
except ZeroDivisionError:
    # Code
except TypeError:
    # Code
except:
    # All other exceptions
```

Why multiple excepts?

Helps handle specific errors differently and keeps code clean and readable.



3. Raising Exceptions

```
age = -5
if age < 0:
    raise ValueError("Age cannot be negative")
```

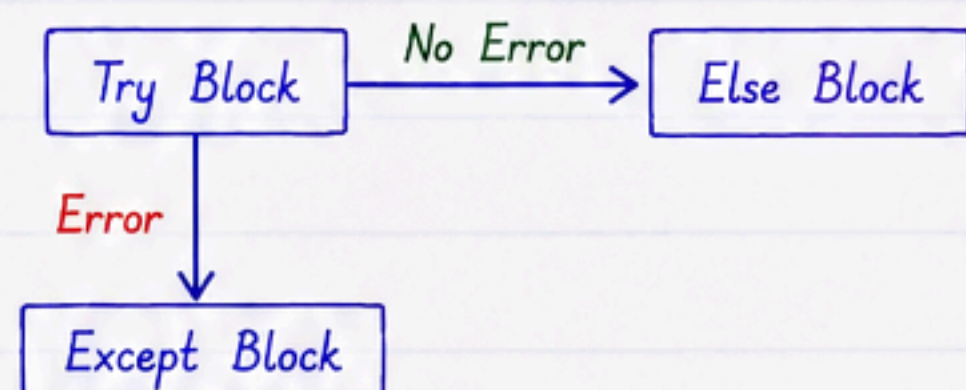
When to raise?

Use raise to signal that something is wrong in your program.



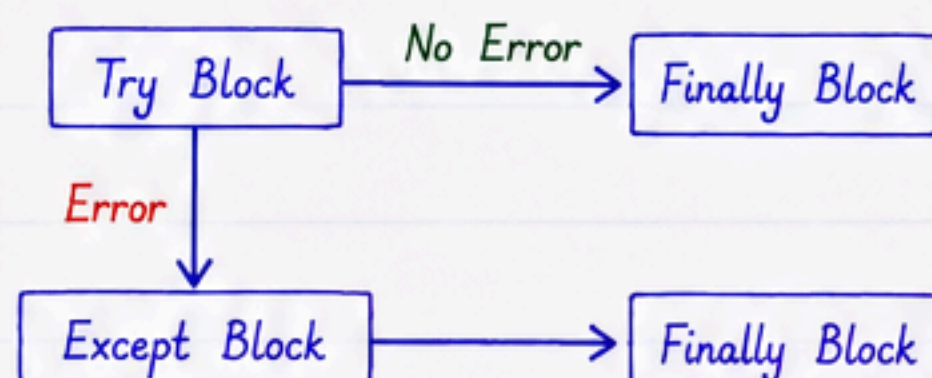
4. Try With Else Clause

```
try:
    # Some code
except:
    # Some code
else:
    # Executed if try was successful
```



5. Try With Finally

```
try:
    # Some code
except:
    # Some code
finally:
    # Executed regardless of error
```



If `--name--` == '`--main--`' In Python

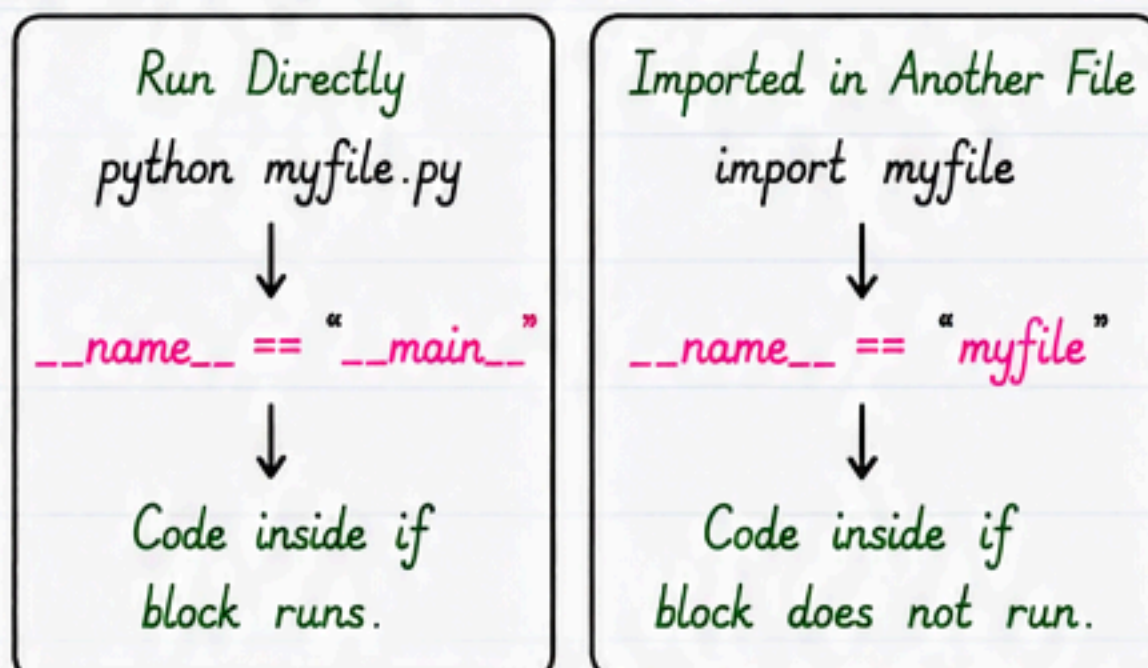
- '`--name--`' evaluates to the name of the module in python from where the program is ran.
- If the module is being run directly from the command line, the '`--name--`' is set to string "`--main--`".
- Thus, this behaviour is used to check whether the module is run directly or imported to another file.

```
if __name__ == "__main__":  
    print("Running directly")
```

It helps in writing code that runs only when the file is executed directly!



How it works?

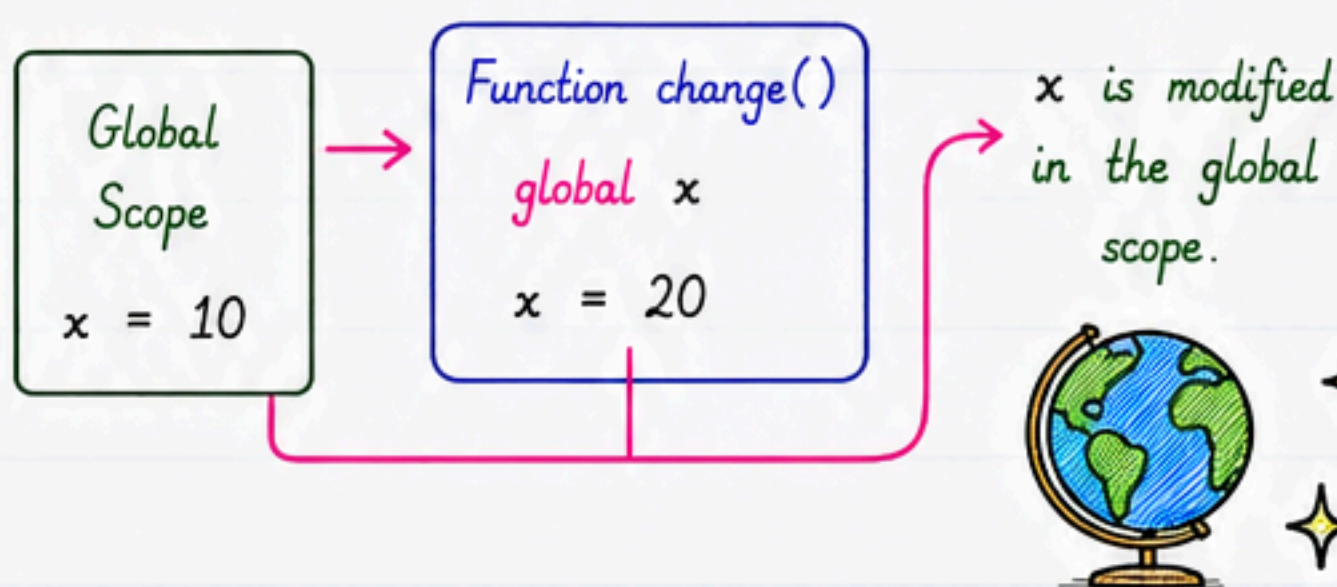


The Global Keyword

- '`global`' keyword is used to modify the variable outside of the current scope.

```
x = 10  
def change():  
    global x  
    x = 20
```

How it works?



Enumerate Function In Python

- The '`enumerate`' function adds counter to an iterable and returns it.

```
list1 = ["Harry", "Rohan", "Shubham"]  
for i, item in enumerate(list1):  
    print(i, item)
```

Example Output

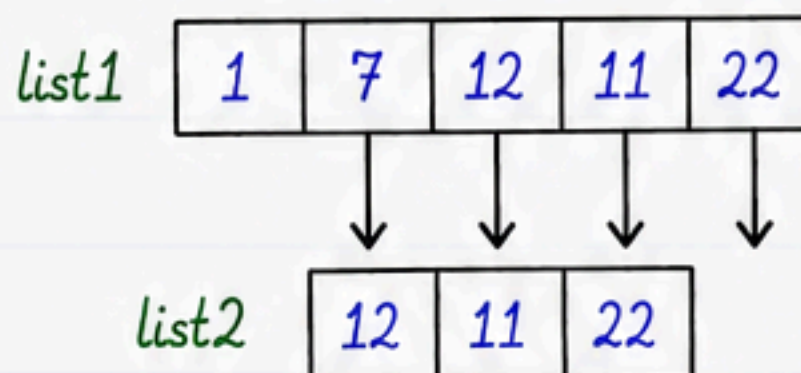
Index (i)	Item
0	Harry
1	Rohan
2	Shubham

Returns a sequence of (index, item) pairs.

List Comprehensions

- List Comprehension is an elegant way to create lists based on existing lists.

```
list1 = [1, 7, 12, 11, 22]  
list2 = [item for item in list1 if item > 8]
```



Includes only items that satisfy the condition.



Code is like poetry,
Write it clean, make it
simple and powerful! ❤️



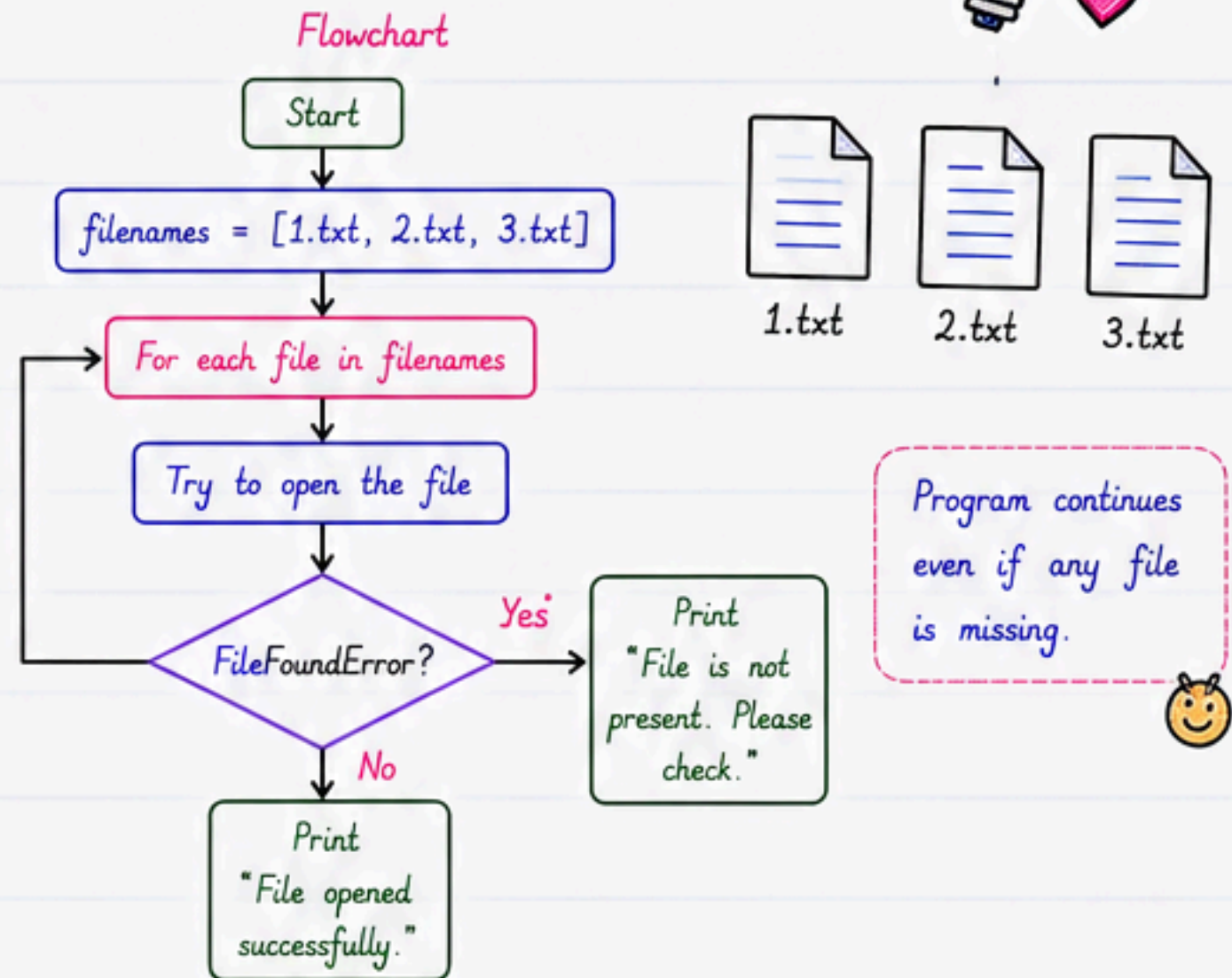


Practice Set



1. Write a program to open three files 1.txt, 2.txt and 3.txt. If any of these files are not present, a message without exiting the program must be printed prompting the same.

```
filenames = ["1.txt", "2.txt", "3.txt"]
for file in filenames:
    try:
        f = open(file, "r")
        print(f"{file} opened successfully.")
        f.close()
    except FileNotFoundError:
        print(f"{file} is not present. Please check.")
```



2. Write a program to print third, fifth and seventh element from a list using enumerate function.

```
mylist = [10, 20, 30, 40, 50, 60, 70, 80, 90]
for i, item in enumerate(mylist):
    if i in [2, 4, 6]:
        print(f"Element at index {i} : {item}")
```

How it works?

Index (i)	Item
0	10
1	20
2	30 ← print (third)
3	40
4	50 ← print (fifth)
5	60
6	70 ← print (seventh)
...	...



Indices start from 0, so third, fifth and seventh indices are 2, 4 and 6.

3. Write a list comprehension to print a list which contains the multiplication table of a user entered number.

```
n = int(input("Enter a number: "))
table = [n * i for i in range(1, 11)]
print(table)
```

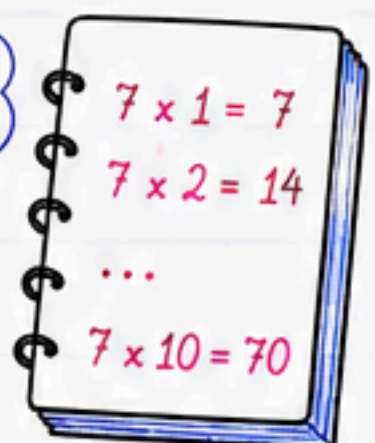
Example

If user enters $n = 7$

Output:

[7, 14, 21, 28, 35, 42, 49, 56, 63, 70]

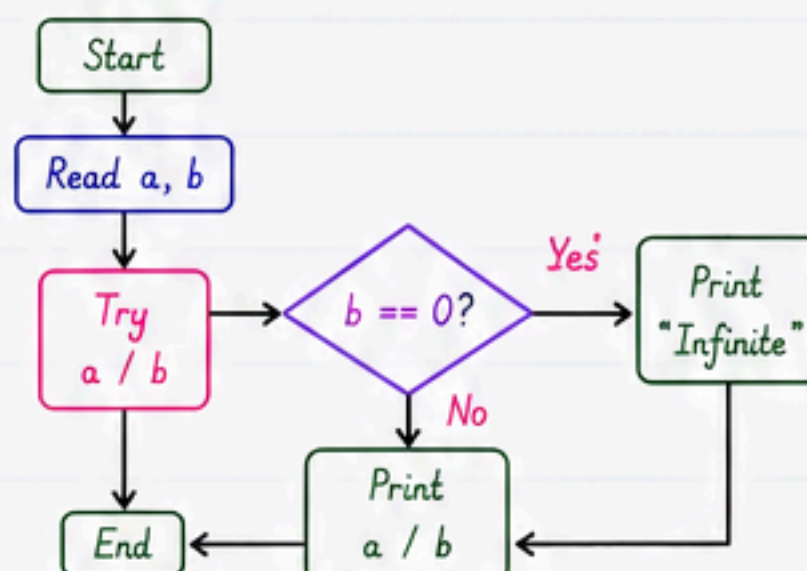
Times Table



4. Write a program to display a/b where a and b are integers. If $b=0$, display infinite by handling the 'ZeroDivisionError'.

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))
try:
    result = a / b
    print(f"a / b = {result}")
except ZeroDivisionError:
    print("Infinite")
```

Flowchart



Handles division by zero without crashing.

5. Store the multiplication tables generated in problem 3 in a file named Tables.txt.

```
n = int(input("Enter a number: "))
table = [n * i for i in range(1, 11)]
with open("Tables.txt", "w") as f:
    for i, val in enumerate(table, start=1):
        f.write(f"{n} x {i} = {val}\n")
    print("Table stored in Tables.txt")
```

Example ($n = 7$)

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```



Each line of the file contains one row of the table.

